

WSL 디스크 사용량 및 유용한 CLI 도구 - 대화 전체 기록

사용자

wsl을 설치하면 이게 Windows의 어디에 설치되고 이후 패키지 설치 등이 되면서 전체적으로 얼마나 디스크의 공간을 차지하는지 확인하는 방법은?

Claude

WSL이 실제로 어디에 얼마만큼의 디스크를 쓰고 있는지 확인하는 방법을 정리해 드리겠습니다.

기본 저장 위치

WSL2 배포판은 하나의 가상 디스크 파일(ext4.vhdx)에 전체 리눅스 파일시스템을 담습니다. apt, pip, npm 등으로 설치하는 패키지도 전부 이 파일 안에 쌓이기 때문에, 이 파일 하나의 크기만 확인하면 전체 사용량을 알 수 있습니다.

- Microsoft Store 로 설치한 경우 보통:

C:\Users\<사용자명>\AppData\Local\Packages\<배포판
패키지명>\LocalState\ext4.vhdx

- wsl --install 로 설치한 최신 방식(Windows 11 일부 버전)에서는:

C:\Users\<사용자명>\AppData\Local\wsl\<GUID>\ext4.vhdx

- wsl --import 로 커스텀 위치에 설치했다면 그때 지정한 경로

확인 절차 (단계별)

1. **설치된 배포판 목록 확인:** PowerShell에서 `wsl --list --verbose` 를 실행하면 설치된 배포판 이름, 상태, WSL 버전(1 또는 2)을 볼 수 있습니다. 이름을 정확히 알아야 다음 단계에서 파일을 찾기 쉽습니다.
2. **vhdx 파일 위치 자동 검색:** PowerShell에서 `Get-Childitem -Path "$env:LOCALAPPDATA\Packages", "$env:LOCALAPPDATA\wsl" -Filter "ext4.vhdx" -Recurse -ErrorAction SilentlyContinue` 를 실행하면 실제 vhdx 파일 경로들을 찾아줍니다.
3. **파일 크기(디스크 실사용량) 확인:** 찾은 경로에 대해 `Get-Item "경로\ext4.vhdx" | Select Length` 를 실행하면 바이트 단위 크기가 나옵니다. 참고로 vhdx는 '동적 확장' 방식이라 삭제한 용량이 자동으로 줄어들지 않고, 한 번 늘어난 뒤에는 그대로 유지되는 경우가 많습니다.
4. **리눅스 내부에서 실제 사용량 확인:** WSL 배포판 안에서 `df -h /` 를 실행하면 파일시스템

전체 사용량을, `du -sh /* 2>/dev/null | sort -h` 를 실행하면 어느 디렉터리가 용량을 많이 쓰는지 구체적으로 확인할 수 있습니다. apt 캐시나 pip/npm 캐시가 의외로 큰 경우가 많습니다.

5. 불필요하게 커진 **vhdx 압축(선택)**: vhdx 파일이 내부 실사용량보다 훨씬 크다면 배포판을 종료(`wsl --shutdown`)한 후 `diskpart`로 `select vdisk file="경로" → compact vdisk` 명령을 실행하거나, Windows 11 22H2 이상에서는 `wsl --manage <배포판이름> --set-sparse true` 로 자동 압축을 켤 수 있습니다.

핵심 정리

- WSL2 의 모든 파일(배포판 자체 + apt/pip/npm 등으로 설치한 패키지)은 결국 **`ext4.vhdx` 파일 하나**에 저장됩니다.
- 이 파일의 크기가 곧 "WSL 이 디스크에서 차지하는 전체 용량"입니다.
- 다만 vhdx 는 삭제해도 자동으로 줄어들지 않는 특성이 있어서, 실제 사용량과 파일 크기가 크게 벌어질 수 있습니다 — 이때는 마지막 단계의 압축 작업이 필요합니다.

만약 특정 배포판 하나만 완전히 지우고 싶으시면 `wsl --unregister <배포판이름>` 으로 해당 vhdx 파일 자체를 삭제할 수도 있습니다.

사용자

```
C:\Users\Wchois>wsl --list --verbose
```

NAME	STATE	VERSION
* Ubuntu	Running	2
rancher-desktop	Stopped	2
rancher-desktop-data	Stopped	2

Claude

현재 Ubuntu, rancher-desktop, rancher-desktop-data 이렇게 3개의 배포판이 설치되어 있으시네요. rancher-desktop 관련 배포판은 Rancher Desktop(컨테이너/쿠버네티스 툴)이 자체적으로 만든 WSL 배포판이라 일반 Ubuntu와는 별도로 용량을 차지합니다.

해결 방법 1 — 리스트 형식으로 출력 (가장 확실함)

```
Get-ChildItem -Path $paths -Filter "*.vhdx" -Recurse -ErrorAction SilentlyContinue |  
    Select-Object FullName,  
    @{Name="SizeGB";Expression={[math]::Round($_.Length/1GB,2)}} |  
    Format-List
```

사용자

FullName :

C:\Users\Wchois\AppData\Local\Packages\Claude_pzs8srxjxfjc\LocalCache\Roaming\Claude\vm_bundles\claudevms.dle\rootfs.vhdx

SizeGB : 9.74

FullName :

C:\Users\Wchois\AppData\Local\Packages\Claude_pzs8srxjxfjc\LocalCache\Roaming\Claude\vm_bundles\claudevms.dle\sessiondata.vhdx

SizeGB : 0.47

FullName :

C:\Users\Wchois\AppData\Local\Packages\Claude_pzs8srxjxfjc\LocalCache\Roaming\Claude\vm_bundles\claudevms.dle\smol-bin.vhdx

SizeGB : 0.04

FullName : C:\Users\Wchois\AppData\Local\wsl\{b15243d0-705f-496e-8256-371303c820b5}\ext4.vhdx

SizeGB : 5.2

FullName : C:\Users\Wchois\AppData\Local\rancher-desktop\distro\ext4.vhdx

SizeGB : 1.5

FullName : C:\Users\Wchois\AppData\Local\rancher-desktop\distro-data\ext4.vhdx

SizeGB : 6.47

Claude

결과가 잘 나왔습니다. 정리해 드리면:

항목	용도	크기
----	----	----

wslW...Wext4.vhdx	Ubuntu (일반 작업용 WSL)	5.2 GB
rancher-desktopWdistroWext4.vhdx	Rancher Desktop 시스템 배포판	1.5 GB
rancher-desktopWdistro-dataWext4.vhdx	Rancher Desktop 데이터(컨테이너 이미지·볼륨)	6.47 GB
claudevm.bundleWrootfs.vhdx	Claude 앱 코드실행 VM 루트	9.74 GB
claudevm.bundleWsessiondata.vhdx	Claude 앱 VM 세션 데이터	0.47 GB
claudevm.bundleWsmol-bin.vhdx	Claude 앱 VM 바이너리	0.04 GB

WSL(Ubuntu + Rancher Desktop)만 합산: $5.2 + 1.5 + 6.47 = \text{약 } 13.17 \text{ GB}$

Claude 앱 VM까지 포함한 전체 vhdx 합계: $13.17 + 9.74 + 0.47 + 0.04 = \text{약 } 23.42 \text{ GB}$

몇 가지 참고하실 점입니다.

- **Claude 앱의 `claudevm.bundle`**은 Claude Desktop 이 코드 실행(파일 생성, bash 등)을 위해 쓰는 별도의 경량 VM 으로, WSL 배포판 목록(`wsl --list`)에는 나타나지 않지만 같은 vhdx 방식이라 이번 검색에 걸린 것입니다. WSL 과는 별개로 관리됩니다.
- **rancher-desktop-data(6.47GB)**가 가장 클 것으로 예상했는데 예상보다 작게 나왔네요 — 도커 이미지를 많이 안 받으신 것 같습니다.
- 용량을 줄이고 싶으시면 안 쓰는 도커 이미지 정리(`rdctl/docker system prune`) 후 Rancher Desktop 의 `distro-data` vhdx 를 압축하시거나, WSL Ubuntu 는 `wsl --shutdown` 후 `diskpart` 로 `compact` 하시면 됩니다.

사용자

wsl에 `ocrmypdf`, `ripgrep`, `fd-find`, `fdupes`, `rdfind` 등을 설치했다면 이런 툴들을 외장 디스크를 대상으로도 수행할 수 있는거지?

Claude

네, 가능합니다. WSL2는 기본적으로 Windows에 연결된 드라이브들을 파일시스템 안으로 마운트해서 보여주기 때문에, 외장 디스크도 대부분 문제없이 접근할 수 있습니다.

어떻게 접근하나요

- 외장 디스크가 Windows 에서 드라이브 문자(예: E:)로 잡혀 있다면, WSL 안에서는 자동으로 `/mnt/e/` 경로로 마운트되어 보입니다. NTFS 든 exFAT 든 상관없이 됩니다.
- 즉 rg 검색어 `/mnt/e/`, fd 확장자 `/mnt/e/`, ocrmypdf `/mnt/e/문서.pdf` `/mnt/e/출력.pdf`, `fdupes -r /mnt/e/`, `rdfind /mnt/e/` 이런 식으로 그대로 실행하시면 됩니다.

한 가지 예외

외장 디스크가 드라이브 문자 없이 물리 디스크 자체로만 인식되거나(예: 리눅스 전용 ext4로 포맷된 디스크), Windows에서 아예 마운트되지 않는 경우라면 `/mnt/`에 자동으로 안 뜹니다. 이때는 관리자 권한 PowerShell에서:

```
wsl --mount \\W.WPHYSICALDRIVE<번호>
```

로 직접 마운트해줘야 합니다.

속도 관련 유의사항

`/mnt/` 경로는 Windows 파일시스템을 9p 프로토콜로 거쳐가는 방식이라, WSL 네이티브 파일시스템(ext4)보다는 확실히 느립니다.

- `ripgrep`, `fd-find` 정도의 검색은 큰 무리 없이 씁니다.
- `fdupes`, `rdfind` 는 파일 전체를 읽고 해시를 계산해야 해서 외장 디스크 + `drvfs` 조합이면 파일 개수가 많을 경우(수만~수십만 개) 체감상 꽤 오래 걸릴 수 있습니다.
- `ocrmypdf` 는 PDF 하나하나가 CPU 바운드 작업이라 I/O 오버헤드 영향은 상대적으로 적은 편입니다.

파일 개수가 아주 많고 반복적으로 스캔하실 계획이면, 작업 대상만 WSL 네이티브 경로(`~/` 아래 등)로 한 번 복사해두고 처리하는 게 체감 속도상 훨씬 빠릅니다.

사용자

WSL 네이티브 경로(`~/` 아래 등) 이 구체적으로 어디를 말하는거야?

Claude

"WSL 네이티브 경로"는 `~/mnt/` 아래가 아닌, WSL 배포판 자체의 리눅스 파일시스템을 말합니다. 구체적으로는:

- `/home/<사용자명>/` — 사용자 홈 디렉터리, 보통 `~`로 표시됩니다. 예: `/home/chois/`
- 이 경로 아래에 있는 모든 것 (`~/work`, `~/tmp` 등 직접 만든 폴더 포함)

왜 이게 "네이티브"인가요

이 경로는 앞서 확인하신 `C:\Users\Wchois\AppData\Local\Wsl\{...}\ext4.vhdx` 파일 내부에 있는 진짜 리눅스(ext4) 파일시스템입니다. Windows 탐색기 입장에서는 이 안이 직접 안 보이고(`\\wsl$\Ubuntu\home\Wchois` 경로로만 우회 접근 가능), WSL 안에서는 순수 리눅스 파일 I/O로 처리되기 때문에 빠릅니다.

반대로 `/mnt/c/`, `/mnt/e/` 같은 경로는 Windows NTFS/exFAT 파일시스템을 9p 프로토콜로 거쳐가는 방식이라 느립니다.

정리하면

경로	실체	속도
<code>/home/chois/</code> , <code>~/</code>	ext4.vhdx 내부 (리눅스 네이티브)	빠름
<code>/mnt/c/Users/chois/...</code>	Windows C드라이브 (NTFS, 9p 경유)	느림
<code>/mnt/e/...</code>	외장 디스크 (9p 경유)	느림

그래서 앞서 말씀드린 방법은, 예를 들어 외장 디스크에서 처리할 파일들을 먼저 `cp -r /mnt/e/대상폴더 ~/work/` 로 홈 디렉터리 아래로 복사한 뒤 `fdupes`, `rfind` 등을 `~/work/` 대상으로 돌리고, 끝나면 결과만 다시 `/mnt/e/`로 옮기거나 원본을 정리하는 방식입니다. 다만 이 방식은 복사하는 데도 시간이 걸리고 홈 디렉터리(=vhdx 파일) 용량을 그만큼 차지하니, 파일 용량이 크다면 "복사해서 처리"가 오히려 손해일 수도 있습니다 — 이 경우엔 그냥 `/mnt/e/`에서 느리더라도 직접 돌리시는 게 나을 수 있습니다.

사용자

복사하는게 나야 그냥 외장 디스크에서 바로 수행하는게 나야?

Claude

상황에 따라 다른데, 판단 기준을 정리해 드리겠습니다.

그냥 외장 디스크(`/mnt/e/`)에서 바로 수행하는 게 나은 경우

- 한 번만 스캔하고 끝낼 작업 (예: 중복 파일 정리를 한 번 하고 다시 안 할 예정)

- 외장 디스크 데이터 용량이 **WSL vhdx** 가 있는 **C 드라이브 여유 공간보다 크거나 비슷한 경우** — 복사 자체가 불가능하거나 C 드라이브를 꽉 채울 위험
- 파일 개수가 그리 많지 않은 경우 (수천~수만 개 수준) — 9p 오버헤드가 있어도 체감상 참을 만함
- ocrmypdf 처럼 **CPU 바운드 작업** — I/O 오버헤드 영향이 상대적으로 작아서 굳이 복사할 이유가 적음

복사해서 `~/` 아래에서 처리하는 게 나은 경우

- 반복적으로 여러 번 스캔/처리할 예정 (한 번 복사해두면 이후 작업은 전부 빠름)
- 파일 개수가 수십만 개 이상이고, fdupes 나 rdfind 처럼 **파일 전체를 읽어 해시 계산**하는 I/O 바운드 작업 — 9p 오버헤드가 누적되면 체감 차이가 꽤 큼
- 처리 대상이 C 드라이브 여유공간보다 확실히 작은 경우

실용적인 절충안

전체를 복사하지 말고, **먼저 외장 디스크에서 그냥 한번 스캔을 돌려보시는 걸 추천드립니다.** fdupes, rdfind는 파일 크기 비교 → 부분 해시 → 전체 해시 순으로 필터링해서 도는 방식이라, 생각보다 느리지 않을 수 있습니다. 실제로 돌려보시고 "이건 너무 느리다" 싶을 때만 그때 복사 전략으로 바꾸시는 게 시간 낭비를 줄이는 방법입니다.

즉 결론: 일단 **`/mnt/e/`에서 바로 시도** → 체감상 너무 느리면 그때 필요한 하위 폴더만 부분 복사, 이 순서를 권해드립니다.

사용자

wsl에서 지원하는 ocrmypdf, ripgrep, fd-find, fdupes, rdfind 등은 실제 유사한 기능을 제공하는 윈도우즈 어플들이 없거나 유료인 경우가 많아서 많이 유용할거 같아. 이것들처럼 실제 PC에서 유용하게 사용할 수 있는 패키지들을 더 추천해 줘봐

Claude

좋은 관점이지네요. 실제로 리눅스 생태계에는 Windows에서 유료거나 아예 없는 기능을 무료로 제공하는 CLI 툴이 많습니다. 카테고리별로 실용적인 것들을 추천해 드리겠습니다.

파일/텍스트 검색·처리

- bat — cat 의 상위호환, 문법 하이라이팅과 줄번호를 붙여서 코드/설정파일 볼 때 편합니다
- fzf — 퍼지 파인더. 파일, 히스토리, 프로세스 등 뭐든 인터랙티브하게 필터링 가능 (다른 툴들과 조합하면 강력함)
- sd — sed 의 직관적인 대체제, 정규식 치환이 훨씬 쉬움
- jq — JSON 파싱/가공 필수 도구 (API 응답, 설정파일 다룰 때)
- yq — jq 의 YAML 버전

미디어/이미지 처리

- ffmpeg — 영상/오디오 변환·자르기·압축 등 사실상 못하는 게 없음, Windows 유료 변환 프로그램 다수 대체 가능
- imagemagick (convert, magick) — 이미지 일괄 리사이즈, 포맷변환, 워터마크 등
- exiftool — 사진/영상 메타데이터(EXIF, GPS 등) 조회·수정·일괄삭제 (사생활 정보 제거용으로도 유용)
- handbrake-cli — HandBrake 의 CLI 버전, 영상 인코딩 배치 작업에 좋음

PDF/문서

- pdftk 또는 qpdf — PDF 병합/분할/암호화 해제 등 (ocrmypdf 와 궁합 좋음)
- pandoc — 문서 포맷 변환 끝판왕 (markdown ↔ docx ↔ pdf ↔ html 등 뭐든 변환)

압축/아카이브

- 7z (p7zip) — 웬만한 포맷 다 풀고 압축 가능, 특히 배치 작업에 스크립팅하기 좋음
- zstd — 최신 고속 압축 알고리즘, 백업 스크립트에 넣으면 gzip 보다 훨씬 빠름

동기화/백업

- rsync — 증분 백업, 대용량 파일 동기화의 표준 (외장 디스크 백업 자동화에 최적)
- restic 또는 borgbackup — 중복제거 + 암호화 백업, 유료 백업 프로그램 대체 가능

시스템 모니터링

- btop 또는 htop — 작업관리자보다 훨씬 상세한 실시간 리소스 모니터링

- `ncdu` — 디스크 용량 차지하는 폴더를 트리 형태로 빠르게 파악 (앞서 `vhdx` 용량 확인하신 작업과 비슷한 용도로 유용)
- `dust` — `du` 의 시각화 버전, `ncdu` 보다 더 직관적

네트워크

- `httpie` — `curl` 보다 사용하기 쉬운 API 테스트 도구

이 중에서 지금 하고 계신 작업 흐름(외장 디스크 정리, PDF/문서 처리)을 감안하면 특히 ``ncdu``(용량 파악), ``rsync``(백업), ``pandoc``, ``exiftool`` 이 바로 체감될 것 같습니다. 설치하는 대부분 `sudo apt install <이름>` 한 줄로 끝납니다.

설치 및 사용

```
sudo apt install bat
batcat /etc/os-release # 이렇게 실행
```

`bat`이라고 짧게 치고 싶으시면 `alias`나 `symlink`를 걸어주면 됩니다:

```
mkdir -p ~/.local/bin
ln -s /usr/bin/batcat ~/.local/bin/bat
```

(`~/.local/bin`이 `PATH`에 잡혀 있다는 전제하에, 이후로는 `bat` 파일명으로 바로 쓰실 수 있습니다.)

사용자

wsl에 현재 설치된 모든 패키지를 다 조회하려면?

Claude

WSL(Ubuntu 등 Debian 계열) 안에서 `apt`로 설치된 패키지 전체 목록을 조회하는 방법입니다.

전체 설치된 패키지 목록 보기

```
dpkg -l
```

패키지명, 버전, 설명이 나오지만 시스템 기본 패키지까지 전부 포함되어 매우 길게 나옵니다.

패키지명만 간단히 보기

```
dpkg-query -W -f='${Package}\n' | sort
```

내가 명시적으로 설치한 패키지만 보기 (추천)

시스템 의존성으로 자동 설치된 패키지는 빼고, 직접 apt install로 설치한 것만 보고 싶다면:

```
apt-mark showmanual | sort
```

이게 실질적으로 가장 유용합니다 — 앞서 설치하신 ripgrep, fd-find, ocrmypdf 같은 것들만 깔끔하게 나옵니다.

snap으로 설치한 패키지가 있다면

```
snap list
```

pip(파이썬), npm(node)으로 설치한 것까지 확인하고 싶다면

```
pip list          # 또는 pip3 list
pip list --user    # 사용자 범위만
npm list -g --depth=0  # 전역 npm 패키지
```

패키지별 크기까지 함께 보고 싶다면

```
dpkg-query -Wf '${Installed-Size}\t${Package}\n' | sort -n | tail -30
```

용량이 큰 순서로 상위 30개를 보여줘서, 디스크 정리할 때 특히 유용합니다.

사용자

pdftk 또는 qpdf 라고 했는데 둘이 뭐가 다른거야? 둘 다 설치해도 돼?

Claude

두 툴 다 PDF를 다루긴 하지만 **성격이 다릅니다**. 같이 설치해도 전혀 문제없고, 오히려 서로 보완적이라 실무에서는 둘 다 깔아두는 경우가 많습니다.

핵심 차이

	pdftk	qpdf
성격	"사용자 친화적" PDF 도구 (병합/분할/폼 조작 위주)	"구조 변환" 저수준 라이브러리형 도구
주 용도	병합, 분할, PDF 폼 채우기 같은 다양한 작업	PDF 파일에 대한 구조적이고 내용을 보존하는 변환 작업, 웹 최적화(linearize), 암호화
구현	Java 기반(요즘 배포판은 pdftk-java)	C++ 기반, 별도 런타임 불필요
Ubuntu 설치	최신 버전에서는 기본 저장소에서 바로 안 되는 경우가 있어 PPA나 snap이 필요할 수 있음	모든 주요 배포판 저장소에 포함되어 있어 바로 설치 가능
개발 활발도	상대적으로 개발이 뜸함	더 활발하게 유지보수됨

실질적으로 뭘 쓰게 되나

- **`pdftk`**: 여러 PDF 합치기, 페이지 뽑아내기, 폼 필드 채우기/추출, 페이지 회전 등 "PDF 를 재구성"하는 작업에 직관적입니다. 명령어도 `pdftk 1.pdf 2.pdf cat output merged.pdf` 처럼 단순합니다.
- **`qpdf`**: 암호 해제/암호화, PDF 손상 복구, 웹용 최적화(linearize), 압축 스트림 풀어서 텍스트 에디터로 열어보기 등 "PDF 내부 구조를 다루는" 작업에 강합니다.

같이 설치해도 되나요

네, 완전히 독립적인 프로그램이라 **충돌 없이 둘 다 설치** 가능합니다.

```
sudo apt install qpdf
sudo apt install pdftk # 안 되면 pdftk-java 시도, 그래도 안 되면 PPA 필요
```

한 가지 참고하실 점은, **Ubuntu 최신 버전에서는 `pdftk`가 기본 저장소에 없을 수 있어서** 안 되면 `sudo apt install pdftk-java`로 시도해보시고, 그마저 안 되면 서드파티 PPA(ppa:malteworld/ppa)를 추가해야 할 수도 있습니다. 반면 **`qpdf`는 표준 저장소에 항상 있어서** 설치가 간편합니다.

실전 팁을 드리면, 단순 병합/분할/폼 작업이 많으시면 pdftk 위주로, 암호 걸린 PDF를 풀거나 손상된 PDF를 복구하는 일이 잦으시면 qpdf가 더 유용할 겁니다. 둘 다 설치해두고 상황에 맞게 쓰시는 걸 추천드립니다.

사용자

```
chois@BOOK-JGU1P4BOIM:/mnt/c/Users/chois$ sudo apt install dust -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
No apt package "dust", but there is a snap with that name.
Try "snap install dust"
E: Unable to locate package dust
```

이제 dust 설치 (컴파일 없이 바로)

```
cd ~
curl -LO https://github.com/bootandy/dust/releases/download/v1.2.4/du-dust_1.2.4-1_amd64.deb
sudo dpkg -i du-dust_1.2.4-1_amd64.deb
dust --version
```

이 방법은 이미 컴파일된 바이너리를 그냥 설치하는 거라 몇 초 안에 끝나고, 앞서 겪으신 컴파일러 문제도 전혀 발생하지 않습니다.

사용자

```
chois@BOOK-JGU1P4BOIM:/mnt/c/Users/chois$ cd ~
curl -LO https://github.com/bootandy/dust/releases/download/v1.2.4/du-dust_1.2.4-1_amd64.deb
sudo dpkg -i du-dust_1.2.4-1_amd64.deb
dust --version

% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
```

```

          Dload Upload  Total  Spent  Left  Speed
  0   0   0   0   0   0   0   0  --:--:-- --:--:-- --:--:--    0
100 969k 100 969k   0   0 1336k   0  --:--:-- --:--:-- --:--:-- 10.9M
[sudo] password for choi:
Selecting previously unselected package du-dust.
(Reading database ... 64564 files and directories currently installed.)
Preparing to unpack du-dust_1.2.4-1_amd64.deb ...
Unpacking du-dust (1.2.4-1) ...
Setting up du-dust (1.2.4-1) ...
Processing triggers for man-db (2.12.0-4build2) ...
-bash: /home/choi/.cargo/bin/dust: No such file or directory
choi@BOOK-JGU1P4BOIM:~$
```

된거야?

Claude

패키지 설치 자체는 성공했습니다. 문제는 셸이 아직 예전 PATH 캐시(~/.cargo/bin/dust)를 기억하고 있어서 엉뚱한 위치를 찾으려는 것뿐입니다. du-dust 패키지가 만드는 실제 실행파일은 /usr/bin/dust에 설치됩니다.

해결 방법

셸의 명령어 캐시를 새로고침하시면 됩니다:

```
hash -r
dust --version
```

또는 그냥 터미널을 껐다 새로 여시거나, 새 WSL 세션을 하나 열어서 dust --version을 실행하셔도 동일하게 해결됩니다.

원인

앞서 rustup으로 Rust를 설치하셨을 때 ~/.cargo/bin이 PATH에 먼저 등록되면서, bash가 dust라는 이름을 그 경로에 있는 것으로 기억(해시)해뒀던 겁니다. rustup을 제거하면서 ~/.cargo/bin/dust 파일 자체는 없어졌지만(애초에 컴파일 실패해서 생기지도 않았을 겁니다), bash의 내부 캐시는 그대로 남아있던 상황입니다. hash -r이 이 캐시를 지워서 PATH를 다시 처음부터 검색하게 만듭니다.

dust --version이 정상적으로 버전을 출력하면 완전히 설치된 겁니다.

사용자

```
chois@BOOK-JGU1P4BOIM:~$ hash -r
dust --version
Dust 1.2.4
chois@BOOK-JGU1P4BOIM:~$
```

Claude

완벽합니다. Dust 1.2.4 출력되면 정상적으로 설치 완료된 겁니다.

이제 아래처럼 바로 써보실 수 있습니다.

```
dust          # 현재 디렉터리 용량 트리 보기
dust /mnt/e   # 외장 디스크 대상
dust -d 2 ~   # 홈 디렉터리를 depth 2까지만
```

du나 ncdu보다 훨씬 직관적으로 어느 폴더가 용량을 많이 먹는지 막대그래프 형태로 바로 보여줄 겁니다.